

Programming Languages and Environments (Lecture 20)

LEI - Licenciatura em Engenharia Informática

João Costa Seco (joao.seco@fct.unl.pt)

Agenda

- Language support for concurrency (part 2)
 - Java
 - OCaml 5.0
 - Kotlin
 - Go
 - Rust

Concurrency in Java

Monitors

- Every object in Java is a monitor, **synchronized** gives access to a mutex and maintains a list of waiting threads..

```
class Counter {  
    private int count = 0;  
  
    public synchronized void increment() {  
        count++;  
    }  
  
    public synchronized int get() {  
        return count;  
    }  
}
```

Monitors

- Every object in Java is a monitor, **synchronized** gives access to a mutex and maintains a list of waiting threads..

```
class BoundedCounter {  
    private int count;  
  
    public BoundedCounter(int initial) { this.count = initial; }  
  
    public synchronized void increment() {  
        count++;  
        notifyAll(); // wake any waiting decrementers  
    }  
  
    public synchronized void decrement() throws InterruptedException {  
        while (count == 0)  
            wait(); // release monitor and sleep until notified  
        count--;  
    }  
  
    public synchronized int get() { return count; }  
}
```

Concurrency in OCaml 5

Domains

- Domains map to CPU threads, have their own heap and run a separate GC.

```
let d = Domain.spawn (  
    fun () -> compute_something ()  
) in  
let result = Domain.join d in  
Printf.printf "result: %d\n" result
```

Domains

- Domains map to CPU threads, have their own heap and run a separate GC.

```
let parallel_map f xs =  
  let domains = List.map (fun x -> Domain.spawn (fun () -> f x)) xs in  
  List.map Domain.join domains
```

```
let () =  
  let results = parallel_map is_prime [1000003; 1000033; 1000037] in  
  List.iter (Printf.printf "%b ") results
```

```
let map_reduce f g xs =  
  let ys = parallel_map f xs in g ys
```

```
let total_words lines =  
  map_reduce count_words (List.fold_left (+) 0) lines
```


Effects

- Like exceptions that can be resumed.
- Tasks can be enqueued and yield control on their own, allowing others to proceed.
- Basis for Fibers, lightweight collaborative threads
- Basis for coroutines in languages like Kotlin
- Effects are GADT (advanced)

```
type _ Effect.t += Yield : unit Effect.t

let two_steps () =
  print_endline "step 1";
  Effect.perform Yield;
  print_endline "step 2"

let run_paused f =
  Effect.Deep.match_with f ()
  { retc = Fun.id
  ; exnc = raise
  ; effc = fun (type a) (e : a Effect.t) ->
      match e with
      | Yield -> Some (fun k ->
          print_endline "-- paused --";
          Effect.Deep.continue k ())
      | _ -> None }

let () = run_paused two_steps
(* step 1
   -- paused --
   step 2      *)
```

Atomic & Mutex

```
let counter = Atomic.make 0

let d1 = Domain.spawn (fun () ->
  for _ = 1 to 100000 do
    Atomic.incr counter
  done)

let d2 = Domain.spawn (fun () ->
  for _ = 1 to 100000 do
    Atomic.incr counter
  done)

Domain.join d1; Domain.join d2;
Printf.printf "count: %d\n" (Atomic.get counter)
```

```
let mu = Mutex.create ()
let shared = ref []

let push_safe x =
  Mutex.lock mu;
  shared := x :: !shared;
  Mutex.unlock mu
```

Coroutines in Kotlin

Coroutines

- All coroutines launched inside a `runBlocking` element can voluntarily interleave until all reach the end. Each coroutine is transformed into a state machine with all possible interleaving states visible to the outside scheduler context.

```
import kotlinx.coroutines.*

fun main() = runBlocking {
    launch {
        yield()           // suspend
        println("world")
    }
    println("hello")
}
```

Goroutines

Goroutines

- The Go runtime distributes goroutines by real threads with their own stack.

```
package main

import (
    "fmt"
    "time"
)

func main() {
    go fmt.Println("hello from goroutine")
    time.Sleep(10 * time.Millisecond)
}
```


Channel (blocking message passing)

- Go's concurrency model is directly inspired by Hoare's CSP model. Goroutines are cheap, channels are typed, and select enables non-deterministic multi-channel communication.

```
ch := make(chan int)

go func() {
    ch <- 42    // send (blocks until received)
}()

v := <-ch      // receive (blocks until send)
fmt.Println(v) // 42
```

Channel (blocking message passing)

- Go's concurrency model is directly inspired by Hoare's CSP model. Goroutines are cheap, channels are typed, and select enables non-deterministic multi-channel communication.

```
func fanIn(ch1, ch2 <-chan string) <-chan string {  
    merged := make(chan string)  
    go func() {  
        for {  
            select {  
            case v := <-ch1: merged <- v  
            case v := <-ch2: merged <- v  
            }  
        }  
    }()  
    return merged  
}
```


Async/Await in Rust

Threads

- The standard thread library of Rust.

```
use std::thread;

fn main() {
    let handle = thread::spawn(|| { println!("Hello from thread"); 42 });

    let result = handle.join().unwrap();
    println!("Thread returned: {result}");
}
```

Threads with Tokio

- Tokio is an executor of asynchronous tasks. Maintains a pool of worker threads

```
#[tokio::main]
async fn main() {
    // Spawn two concurrent async tasks
    let t1 = tokio::task::spawn(fetch_url("https://example.com"));
    let t2 = tokio::task::spawn(fetch_url("https://example.org"));

    // Join both – like Promise.all
    let (r1, r2) = tokio::join!(t1, t2);
    println!("both done");
}
```


Threads with Tokio

- Tokio is an executor of asynchronous tasks. Maintains a pool of worker threads

```
#[tokio::main]
async fn main() {
    let mut t1 = tokio::task::spawn(fetch_url("https://example.com"));
    let mut t2 = tokio::task::spawn(fetch_url("https://example.org"));

    // select! completes as soon as the first task finishes, or times out
    tokio::select! {
        result = &mut t1 => println!("example.com finished first: {:?}", result),
        result = &mut t2 => println!("example.org finished first: {:?}", result),
        _ = tokio::time::sleep(Duration::from_secs(2)) => println!("timed out"),
    }
}
```